

APPLIED RESEARCH

Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation

SHANTHI KARPURAPU¹, SRAVANTHY MYNENI², UNNATI NETTUR³,
LIKHIT SAGAR GAJJA⁴, DAVE BURKE⁵, TOM STIEHM⁵, AND JEFFERY PAYNE⁵

¹Digital Innovation, SQAC, Ashburn, VA 20147, USA

²Department of Information Technology and Management, Illinois Institute of Technology, Chicago, IL 60616, USA

³Department of Computer Science, Virginia Tech, Blacksburg, VA 24061, USA

⁴Department of Computer Science, BML Munjal University, Gurugram, Haryana 122413, India

⁵Coveros Inc., Fairfax, VA 22033, USA

Corresponding author: Shanthi Karpurapu (shanthi.karpurapu@gmail.com)

ABSTRACT Behavior-driven development (BDD) is an Agile testing methodology fostering collaboration among developers, QA analysts, and stakeholders. In this manuscript, we propose a novel approach to enhance BDD practices using large language models (LLMs) to automate acceptance test generation. Our study uses zero and few-shot prompts to evaluate LLMs such as GPT-3.5, GPT-4, Llama-2-13B, and PaLM-2. The paper presents a detailed methodology that includes the dataset, prompt techniques, LLMs, and the evaluation process. The results demonstrate that GPT-3.5 and GPT-4 generate error-free BDD acceptance tests with better performance. The few-shot prompt technique highlights its ability to provide higher accuracy by incorporating examples for in-context learning. Furthermore, the study examines syntax errors, validation accuracy, and comparative analysis of LLMs, revealing their effectiveness in enhancing BDD practices. However, our study acknowledges that there are limitations to the proposed approach. We emphasize that this approach can support collaborative BDD processes and create opportunities for future research into automated BDD acceptance test generation using LLMs.

INDEX TERMS Agile software development, natural language processing, prompt engineering, software testing, test case automation, automated acceptance testing, zero-shot and few-shot, GPT-3.5 and GPT-4, PaLM-2, Llama-13B, cucumber, generative AI.

I. INTRODUCTION

BDD is a widely adopted, test-first Agile testing practice for specifying system behavior through tests. It promotes collaboration among software developers, quality assurance analysts, and business stakeholders [1], [2], [3]. BDD utilizes the Gherkin language [4], [5], a semi-structured natural language, to express the acceptance criteria of user stories in

human-readable acceptance tests written as scenarios framed by the Given, When, Then keywords [6].

A simple scenario is provided in Table 1 to illustrate the use of the Gherkin language. In Table 1, The Given step establishes the initial condition, ensuring the calculator is ready for use. The When step represents the action – entering the numbers 5 and 7 and choosing addition. Finally, the Then step defines the expected outcome – the calculator displays the result as “12”. This Gherkin structure enhances clarity and enables non-technical professionals to understand scenarios.

The associate editor coordinating the review of this manuscript and approving it for publication was Longzhi Yang¹.

TABLE 1. BDD scenario example.

Scenario: Performing Addition
Given I have opened the calculator application
When I enter “5” into the calculator
And I add “7”
Then the result should be “12”

Despite its effectiveness in translating software specifications into behavior for validation, BDD experiences challenges in manually crafting acceptance tests, leading to a notable impact on efficiency and accuracy [7]. Ensuring high-quality BDD acceptance tests is crucial for successful implementation [8]. The term “high-quality BDD acceptance tests” denotes adherence to Gherkin rules and best practices outlined in the methodology section [2]. In addition, significant experience is required to craft good quality BDD acceptance tests.

In this manuscript we describe an approach that optimizes BDD practices using LLMs, aiming to automate acceptance test generation, reduce manual effort, and enhance productivity. The suggested approach can also aid in standardizing the BDD acceptance test formulation in Agile teams. Our study explores the automation of BDD acceptance test possibilities using LLMs such as GPT-3.5, GPT-4, Llama-2-13B, and PaLM-2. Additionally, it delves into the significance of prompt engineering for evaluating and selecting optimal prompts to ensure the generated scenarios adhere to best practices and avoid Gherkin syntax issues.

II. RELATED WORKS

This section provides an overview of our literature review on BDD acceptance test creation automation. Despite our exhaustive search, we found no prior research in this field. Therefore, we broadened our literature review to explore the automation of conventional test case generation methods. The term ‘conventional test case generation method’ refers to test cases written with preconditions, test steps, expected results, and actual results. We have reviewed approaches that used conventional based methods.

Mathur et al. [9] proposed a method for generating test cases from conversation text. The context and topic are extracted from the conversation text using T5 and GPT-3 models. This approach eliminates the need for manual test case creation and enables the testing of complex systems. However, the method may need to enhance its performance when generating numerous test cases and exhibiting accuracy issues in the output.

Zhuang et al. [10] proposed a technique in 2018 for generating short test cases using an ensemble model. This method uses multiple rankers and a generation-based approach to produce the final response, which is selected based on the ensemble module ranking provided. However, Interpreting the output of an ensemble model can be challenging due to its complexity. Additionally, creating an ensemble model can be

challenging and time-consuming, which may result in lower prediction accuracy compared to using a single model.

In a different study conducted by Lafi et al. [11] proposed an approach to generate test cases automatically from requirement specifications. This approach has a sequence of steps: extracting the required information from the use case description obtained from the use case diagram, then developing the control flow graph and NLP table, and generating test paths as the next step. The test paths and NLP table generate test cases, but there is a need to explore the applicability of the proposed approach to various case studies. This approach does not address validating results obtained using different test coverage criteria.

Guo et al. [12] introduced a framework for generating test cases based on the generative adversarial network (GAN). This study has shown that the WGAN-GP model effectively improves test coverage. The results indicate that unit testing has better test coverage than integration testing. However, the study highlights the potential for enhancing integration tests in the future.

Utting et al. [13] developed an approach for efficient test identification and generation from the customer and test execution traces, particularly in web services and API testing. This approach utilizes clustering algorithms to improve testing procedures. This study only presents preliminary results and has yet to determine the method’s robustness.

Our proposed approach uses LLMs like GPT-3.5, GPT-4, or PaLM-2 with optimized prompt design deviating from machine learning-based conventional test case generation methods [14], [15], [16], [17] mentioned in the literature [14]. In the conventional approach, teams typically write test cases in the later stages of software development. Our approach is developed for BDD based agile methodology [18] and enables early testing and validation. Therefore, teams formulate BDD acceptance tests for user stories before progressing from the sprint backlog to the ‘in progress’ state, defining all specifications upfront in a testable format. Additionally, using LLMs in the proposed approach has the advantage of being simple, easily accessible to everyone, and user-friendly. Adopting this proposed approach is relatively straightforward for Agile teams, provided the organization’s policies allow the use of LLMs.

III. METHODOLOGY

Our proposed methodology aims to comprehensively evaluate different LLMs and prompt design techniques to gauge their effectiveness in generating BDD acceptance tests. Figure 1 illustrates the procedural steps for automating BDD acceptance tests and validation.

We developed separate programs for each LLM and executed each program twice to evaluate both few-shot and zero-shot prompts. The implementation is written in Python, tailored individually for each model, and executed within the Google Colab framework [19]. The code developed for all the models is available in the GitHub repository [20].

The initial step involves reading user story descriptions from a CSV file. Each user story description, accompanied by the prompt (prompt is the task instructions to the model) and model parameters (temperature, top_p, max_tokens etc.), are sent in the request body to the model API endpoint. Iterating through all user stories, we store the model's BDD acceptance tests (scenarios) response as a feature file.

After the feature file generation phase, the generated feature files undergo validation using a tool that validates Gherkin syntax, capturing syntax errors for further analysis. We comprehensively assessed LLMs and prompt design techniques for this proposed methodology that integrates automated generation with rigorous validation.

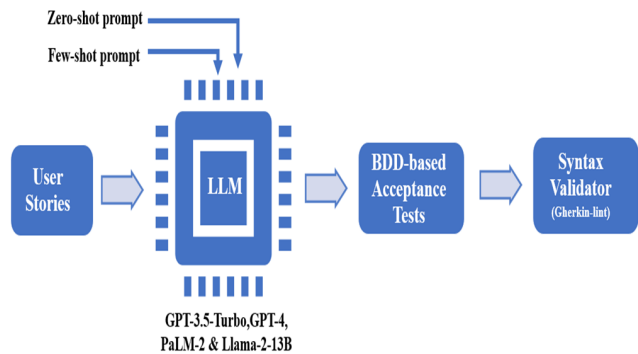


FIGURE 1. Behavior driven development acceptance tests generation and validation process.

The following details outline the essential elements of our proposed methodology.

A. DATASET

For this study, we curated a diverse set of approximately 50 real-world user stories drawn from various sources. The data sources include user story data from Mendeley [21] and a blog post [22] that details user stories across different domains. This compilation ensures a comprehensive representation of user stories for robust examination in our study.

B. PROMPT TECHNIQUES

The prompt technique involves creating clear, concise instructions or queries that guide LLMs in task execution. These prompts are essential because they enhance model comprehension and performance by providing explicit guidance, reducing ambiguity, and ensuring accurate responses. Wei et al. suggested the effectiveness of instruction tuning, which refines models through fine-tuning based on datasets described with clear instructions, thereby enhancing zero-shot learning capabilities [23], [24], [25].

In the context of few-shot learning, prompts are crucial. Despite the impressive zero-shot capabilities of LLMs, they may face challenges with complex tasks. Therefore, few-shot prompting [26] involves incorporating examples within the prompt. This prompt technique enables in-context learning by guiding the model to better performance, conditioning it for subsequent examples with minimal instances.

Adhering to best practices in generating BDD acceptance tests can fully unlock the value of LLM. We evaluated zero and few-shot prompt techniques [23], [27], [28], [29], [30] to determine which prompts better align with best practices. Best practices adoption may vary based on the software development practices followed by the agile teams. Therefore, as part of this study, we considered a subset of common best practices for evaluation, focusing solely on evaluating if the model can give an output based on the best practice instructions given in the prompt.

Table 2 and 3 details the prompt techniques considered in this study. The best practices are included in both the prompts as instructions from 1 to 6, as shown in Table 2 and 3. It is worth noting that zero and few-shot prompts have the same instructions, and the few-shot prompt includes additional BDD scenario examples for the model to follow while producing the response.

TABLE 2. Zero-shot prompt technique instructions.

Generate a feature file with 5 Gherkin Scenarios for {user_story} by following below {instructions}. {instructions}= 1. Start the feature file with the 'Feature:' keyword. 2. Provide a descriptive feature name to specify the context of the scenarios. 3. Include steps in the Background if they are repeated at the beginning of all scenarios in a feature. 4. The background step is executed before every scenario. 5. Use tags as annotations to group and organize scenarios and features. 6. Tags are written with the '@' symbol followed by a significant text. {user_story} = "As a user, I need a simple calculator for quick and accurate basic operations."
--

C. LARGE LANGUAGE MODELS

We listed the LLMs considered for evaluation, as detailed in Table 4. The selected models include:

D. EVALUATION

Manual validation of feature files generated from LLM can be resource-intensive, demanding significant time and effort. In addition, the human-driven nature of the process introduces the potential for biases and errors, emphasizing the need for rigorous checks and collaboration to mitigate these challenges.

In evaluating the quality of the generated feature files, we employed the Gherkin-lint tool [38], [39] to overcome manual validation challenges, a robust instrument designed to detect syntax violations. This tool meticulously evaluates adherence to predefined style guidelines tailored for the Gherkin language. This tool comprehensively evaluates and examines the syntactical correctness of the feature files and ensures alignment with established conventions and best practices in Gherkin-based specifications. This

TABLE 3. Few-shot prompt technique instructions.

User role:
Same instructions from Zero-shot are given along with the examples as written below
Assistant role:
Feature: Basic Calculator Operations
As a user, I need a simple calculator for quick and accurate basic operations.
Background:
Given I have opened the calculator application
@basicoperations
Scenario: Performing Addition
When I enter “5” into the calculator
And I add “7”
Then the result should be “12”
@basicoperations
Scenario: Performing Subtraction
When I enter “10” into the calculator
And I subtract “3”
Then the result should be “7”
Continued.

TABLE 4. Large langue model details.

Model	Details
GPT-3.5-Turbo	This autoregressive LLM utilizes the Reinforcement Learning from Human Feedback (RLHF) mechanism. As the initial backbone model for ChatGPT, GPT-3.5 [32] demonstrates remarkable zero-shot performance across diverse tasks [33]. Our study employs the gpt-3.5-turbo-0613 model from OpenAI, known for its maximum context length of 4K tokens.
GPT-4-Preview	The most recent addition to OpenAI's LLMs, GPT-4, stands out as the first multimodal model in the GPT series [34]. Recognized for its enhanced instruction-following capabilities compared to GPT-3.5, GPT-4 is deemed more reliable. In our study, we utilize the GPT-4 version.
PaLM-2	Developed by Google [35], PaLM-2 is an LLM that employs a mixture of objective techniques It demonstrates substantial performance improvements over the original PaLM model [36]. In our study, we utilize the chat-bison-001 model.
Llama-2 – 13B	Developed by Meta [37], Llama-2 stands out as an open-source LLM. A notable advantage of Llama-2, distinguishing it from the previously mentioned LLMs, is its open-source nature, making it accessible for research and commercial purposes. Our study utilizes the Chat versions of the Llama-2-13B model, sourced from HuggingFace [38].

approach offers a detailed assessment of BDD acceptance tests and enhances the robustness and comprehensiveness of our evaluation process.

IV. RESULTS

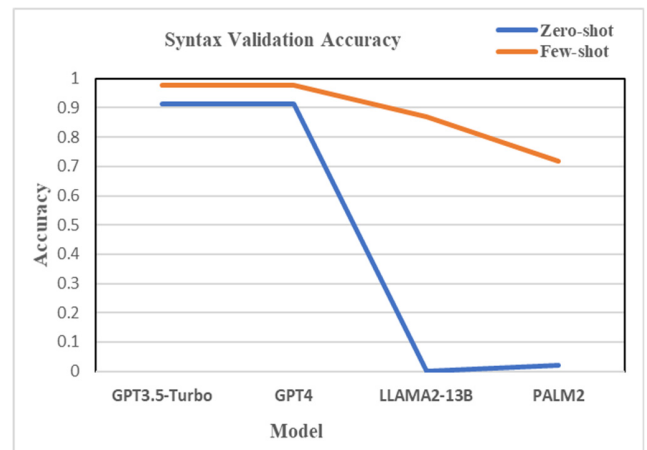
As the methodology section outlines, we generate BDD acceptance tests for all user stories, considering various LLMs and prompts. The subsequent sections present the findings from the conducted experiments. Visual representations have been employed to convey critical findings and offer comprehensive insights into the experiments.

A. VALIDATION ACCURACY

As shown in Equation 1, we calculated accuracy by dividing the number of BDD feature files without syntax issues by the total number of generated BDD feature files. When employing the few-shot prompt technique, we observe a high success rate in generating BDD feature files without syntax issues. GPT-3.5 and GPT-4 models exhibit higher accuracy with the few-shot prompt technique than other models, such as Llama-2-13B and PaLM-2 (Figure 2). Llama-2-13B achieves the second-highest accuracy with the few-shot prompt technique (Figure 2).

Validation Accuracy

$$= \frac{\text{No. of Feature Files w/o Syntax Errors}}{\text{Total No. of Feature Files}} \quad (1)$$

**FIGURE 2.** BDD-based feature syntax validation accuracies across all user stories with various prompt techniques and LLMs.

B. SYNTAX ERRORS

The various syntax errors reported by the Gherkin-lint tool are categorized as shown in Table 5. The aggregation of syntax errors across all models reveals that the highest incidence of errors occurs when employing the zero-shot technique (Figure 3). Notably, the zero-shot approach contributes to 89% of the total syntax errors (Figure 3).

The compilation of syntax errors by error type across all models indicates that the predominant contributors to errors in the zero-shot prompt technique are the “Restricted-Patterns-Present” and “Gherkin-KeyWord-Not-Present” error types. In the few-shot prompt technique, the most prominent error types are “Restricted-Patterns-Present” and “Missing-Tags (Figure 4).

TABLE 5. Syntax error type details.

Error Type	Error Details
gherkin-keywords-not-in-logical-order	When writing scenario steps in some BDD feature files generated by LLM, the tool reports an error if the specified order of scenario step keywords (Given, When, Then) is not followed—for instance, a sequence like "Given Then When" is considered incorrect.
gherkin-keyword-not-present-in-step	The tool reports this error when scenario steps do not commence with the keywords Given, When, Then, And, or But. For instance, inappropriately, tags are sometimes written at the end of the file, such as @regression @download-files @experiments @attachments, violating the correct structure where tags should appear before the scenario description.
missing-tags	The tool reports this error in the BDD feature file when scenarios lack tags. While reporting the lack of tags error is optional in the tool settings, the configuration dictates treating missing tags as an error, as the prompt technique involves generating tags as one of the instructions.
restricted-patterns-present	The presence of unnecessary special characters in the generated BDD feature files triggers the reporting of this error. Examples include: Feature description: "-----" ----" (matches a restricted pattern) Background description: "-----" (matches restricted pattern) Scenario description: "-----"

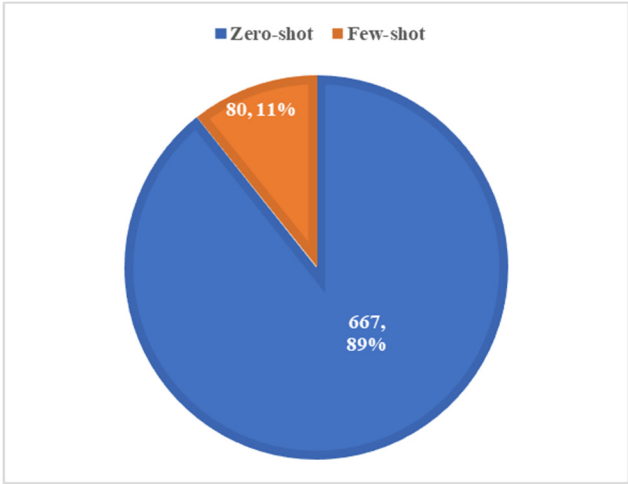


FIGURE 3. Total syntax errors for zero and few-shot techniques.

The examination of model performance concerning syntax errors shown in Figure 5 is a vital aspect. GPT-3.5 and GPT-4 exhibit better performance by showcasing the fewest errors in the few-shot technique. Additionally, the number of syntax issues observed in the zero-shot technique for GPT-3.5 and

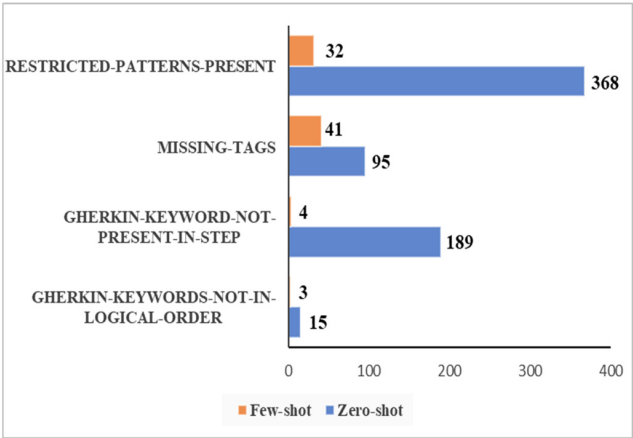


FIGURE 4. Syntax error type distribution.

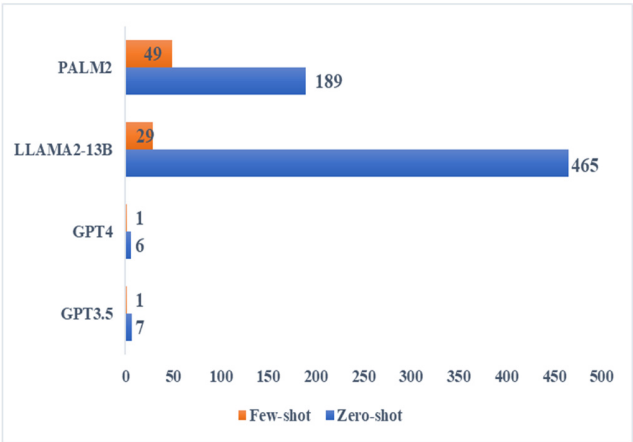


FIGURE 5. No. of syntax errors for various LLMs.

GPT-4 is comparable but slightly higher than that in the few-shot technique. GPT-3.5 and GPT-4 are the most effective models, with fewer syntax errors in generating BDD feature files as shown in Figure 5. Conversely, Llama-2-13B contributes the highest number of errors in the zero-shot technique, while PaLM-2 does so in the few-shot technique.

This comparative analysis evaluates GPT-3.5, GPT-4, Llama-2-13B, and PaLM-2 language models based on their performance in handling Gherkin syntax errors, including gherkin-keywords-not-in-logical-order, gherkin-keyword-not-present-in-step, missing-tags, and restricted-patterns-present. As shown in Table 6 & 7, GPT-3.5 and GPT-4 displayed limited occurrences of errors in zero-shot and few-shot evaluations. Llama-2-13B exhibited challenges during zero-shot evaluations, particularly in gherkin-keyword-not-present-in-step (130 instances) and restricted-patterns-present (335 instances). PaLM-2 demonstrated a spectrum of errors, with many instances involving missing-tags as (95 occurrences). Notably, few-shot evaluations across all models revealed an overall lower number of errors compared to zero-shot evaluations.

Although GPT-3.5 and GPT-4 models performed better with the few-shot prompt technique, our proposed approach

TABLE 6. Syntax error distribution for zero-shot.

Error Type	GPT-3.5	GPT-4	PaLM-2	Llama-2-13B
gherkin-keywords-not-in-logical-order	6	5	4	0
gherkin-keyword-not-present-in-step	1	0	58	130
restricted-patterns-present	0	1	32	335
missing-tags	0	0	95	0

TABLE 7. Syntax error distribution for few-shot.

Error Type	GPT-3.5	GPT-4	PaLM-2	Llama-2-13B
gherkin-keywords-not-in-logical-order	1	1	1	0
gherkin-keyword-not-present-in-step	0	0	3	1
restricted-patterns-present	0	0	4	28
missing-tags	0	0	41	0

had certain limitations within the current research scope. We have manually validated some of the BDD acceptance tests and confirmed the validity of those tests in terms of their applicability to user stories. In our present research scope, we haven't explored the realms of test coverage and the validity of generated tests in terms of their applicability to user stories using any formal validation process. However, we recognize these areas as valuable aspects for future research. The foundational findings from our research should instill confidence and pave the way for a more in-depth examination of test coverage and validity in subsequent research in this field. Our primary aim was to validate the readiness of tests produced by the LLM model, ensuring they are free from syntax issues. Also, we employed a single syntax validation tool for validation. We assumed that the outcomes would likely align across other syntax validator tools.

Furthermore, in our current study, we couldn't utilize any real-time projects due to the lack of dataset, which can boost our confidence if explored with real-time project data. The BDD methodology emphasizes collaboration among stakeholders and developers for crafting acceptance tests. The suggested method should facilitate this collaborative process, signifying its role as a supportive tool. It implies that the approach should assist in the collaborative process and not be utilized in isolation, emphasizing the need for discussions on formulating acceptance tests from the proposed approach.

V. CONCLUSION

Utilizing OpenAI's GPT-3.5 and GPT-4 models and the few-shot prompt technique to create BDD acceptance tests from user stories displays potential for improving productivity in agile software development and implementing standardized BDD practices. Incorporating the few-shot technique, involving instruction tuning with examples has demonstrated improved performance compared to the zero-shot approach. This advancement proves particularly beneficial in addressing complex natural language problems, such as the creation of BDD acceptance tests. The potential success of this approach could inspire further research into utilizing LLMs for automated BDD acceptance test generation to achieve enhanced test coverage.

REFERENCES

- [1] G. Downs, "Lean-agile acceptance test-driven development: Better software through collaboration by Ken Pugh," *ACM SIGSOFT Softw. Eng. Notes*, vol. 36, no. 4, p. 34, 2011.
- [2] L. P. Binamungu, S. M. Embury, and N. Konstantinou, "Characterising the quality of behaviour driven development specifications," in *Proc. Agile Processes Softw. Eng. Extreme Program., 21st Int. Conf. Agile Softw. Develop., XP*, Jun. 2020, pp. 87–102.
- [3] H. M. Abushama, H. A. Allassam, and F. A. Elhaj, "The effect of test-driven development and behavior-driven development on project success factors: A systematic literature review based study," in *Proc. Int. Conf. Comput., Control, Electr., Electron. Eng. (ICCCEEE)*, Khartoum, Sudan, Feb. 2021, pp. 1–9, doi: [10.1109/ICCCEEE49695.2021.9429593](https://doi.org/10.1109/ICCCEEE49695.2021.9429593).
- [4] *Gherkin Syntax*. Accessed: Jan. 24, 2024. [Online]. Available: <https://cucumber.io/docs/>
- [5] K. Nicięja, *Writing Great Specifications: Using Specification by Example and Gherkin*. Shelter Island, NY, USA: Manning, 2017.
- [6] *Gherkin Syntax*. Accessed: Jan. 24, 2024. [Online]. Available: <https://docs.specflow.org/projects/specflow/en/latest/Gherkin/Gherkin-Reference.html/>
- [7] M. S. Farooq, U. Omer, A. Ramzan, M. A. Rasheed, and Z. Atal, "Behavior driven development: A systematic literature review," *IEEE Access*, vol. 11, pp. 88008–88024, 2023, doi: [10.1109/ACCESS.2023.3302356](https://doi.org/10.1109/ACCESS.2023.3302356).
- [8] O. Bezsmertnyi, N. Golian, V. Golian, and I. Afanasieva, "Behavior driven development approach in the modern quality control process," in *Proc. IEEE Int. Conf. Problems Infocommun. Sci. Technol. (PIC S&T)*, Kharkiv, Ukraine, Oct. 2020, pp. 217–220, doi: [10.1109/PICST51311.2020.9467891](https://doi.org/10.1109/PICST51311.2020.9467891).
- [9] A. Mathur, S. Pradhan, P. Soni, D. Patel, and R. Regunathan, "Automated test case generation using T5 and GPT-3," in *Proc. 9th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Coimbatore, India, Mar. 2023, pp. 1986–1992, doi: [10.1109/ICACCS57279.2023.10112971](https://doi.org/10.1109/ICACCS57279.2023.10112971).
- [10] Y. Zhuang, X. Wang, H. Zhang, J. Xie, and X. Zhu, "An ensemble approach to conversation generation," in *Natural Language Processing and Chinese Computing (Lecture Notes in Computer Science)*, Jan. 2018, pp. 51–62, doi: [10.1007/978-3-319-73618-1_5](https://doi.org/10.1007/978-3-319-73618-1_5).
- [11] M. Lafi, T. Alrawashed, and A. M. Hammad, "Automated test cases generation from requirements specification," in *Proc. Int. Conf. Inf. Technol. (ICIT)*, Amman, Jordan, Jul. 2021, pp. 852–857, doi: [10.1109/ICIT52682.2021.9491761](https://doi.org/10.1109/ICIT52682.2021.9491761).
- [12] X. Guo, H. Okamura, and T. Dohi, "Automated software test data generation with generative adversarial networks," *IEEE Access*, vol. 10, pp. 20690–20700, 2022, doi: [10.1109/ACCESS.2022.3153347](https://doi.org/10.1109/ACCESS.2022.3153347).
- [13] M. Utting, B. Legeard, F. Dadeau, F. Tamagnan, and F. Bouquet, "Identifying and generating missing tests using machine learning on execution traces," in *Proc. IEEE Int. Conf. Artif. Intell. Test. (AITest)*, Oxford, U.K., Aug. 2020, pp. 83–90, doi: [10.1109/AITest49225.2020.00020](https://doi.org/10.1109/AITest49225.2020.00020).
- [14] T. Potuzak and R. Lipka, "Current trends in automated test case generation," in *Proc. 18th Conf. Comput. Sci. Intell. Syst. (FedCSIS)*, Warsaw, Poland, Sep. 2023, pp. 627–636, doi: [10.15439/2023F9829](https://doi.org/10.15439/2023F9829).
- [15] F. YazdaniBanafsheDaragh and S. Malek, "Deep GUI: Black-box GUI input generation with deep learning," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2021, pp. 905–916.

- [16] A. Singhal and A. Bansal, "Generation of test oracles using neural network and decision tree model," in *Proc. 5th Int. Conf.-Confluence Next Gener. Inf. Technol. Summit (Confluence)*, Sep. 2014, pp. 313–318.
- [17] J. Zhang, L. Zhang, M. Harman, D. Hao, Y. Jia, and L. Zhang, "Predictive mutation testing," *IEEE Trans. Softw. Eng.*, vol. 45, no. 9, pp. 898–918, Sep. 2019.
- [18] R. Khan, A. K. Srivastava, and D. Pandey, "Agile approach for software testing process," in *Proc. Int. Conf. Syst. Model. Advancement Res. Trends (SMART)*, Moradabad, India, Nov. 2016, pp. 3–6, doi: [10.1109/SYS-MART.2016.7894479](https://doi.org/10.1109/SYS-MART.2016.7894479).
- [19] *Google Colab Infrastructure*. Accessed: Jan. 24, 2024. [Online]. Available: <https://colab.research.google.com/>
- [20] *Github Repository*. Accessed: Feb. 3, 2024. [Online]. Available: [Online]. Available: <https://github.com/karpurapus/BDDGPT-Automate-Tests.git>
- [21] *Mendley-Requirement Data Set (User Stories)*. Accessed: Jan. 24, 2024. [Online]. Available: <https://data.mendeley.com/datasets/7zbk8zsd8y/1>
- [22] *45 User Story Examples to Inspire Your Agile Teams*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.parabol.co/blog/user-story-examples/>
- [23] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. Wei Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," 2021, *arXiv:2109.01652*.
- [24] W. Wang, V. W. Zheng, H. Yu, and C. Miao, "A survey of zero-shot learning," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–37, Jan. 16, 2019, doi: [10.1145/3293318](https://doi.org/10.1145/3293318).
- [25] Y. Xian, C. H. Lampert, B. Schiele, and Z. Akata, "Zero-shot learning—A comprehensive evaluation of the good, the bad and the ugly," 2017, *arXiv:1707.00600*.
- [26] Y. Song, T. Wang, S. K. Mondal, and J. P. Sahoo, "A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities," 2022, *arXiv:2205.06743*.
- [27] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [28] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [29] T. B. Brown et al., "Language models are few-shot learners," 2020, *arXiv:2005.14165*.
- [30] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?" 2022, *arXiv:2202.12837*.
- [31] K. Subramanyam Kalyan, "A survey of GPT-3 family large language models including ChatGPT and GPT-4," 2023, *arXiv:2310.12321*.
- [32] M. Tahmid Rahman Laskar, M. Saiful Bari, M. Rahman, M. Amran Hossen Bhuiyan, S. Joty, and J. Xiangji Huang, "A systematic study and comprehensive evaluation of ChatGPT on benchmark datasets," 2023, *arXiv:2305.18486*.
- [33] J. Achiam et al., "Gpt-4 technical report," 2023, *arXiv:2303.08774*.
- [34] R. Anil et al., "PaLM 2 technical report," 2023, *arXiv:2305.10403*.
- [35] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," 2022, *arXiv:2204.02311*.
- [36] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [37] T. Wolf et al., "HuggingFace's transformers: State-of-the-art natural language processing," 2019, *arXiv:1910.03771*.
- [38] *Gherkin-Lint, Github Repository*. Accessed: Jan. 24, 2024. [Online]. Available: <https://github.com/gherkin-lint/gherkin-lint>
- [39] *Gherkin-Lint, NPM*. Accessed: Jan. 24, 2024. [Online]. Available: <https://www.npmjs.com/package/gherkin-lint>

SHANTHI KARPURAPU received the Bachelor of Technology degree in chemical engineering from Osmania University, Hyderabad, India, and the Master of Technology degree in chemical engineering from the Institute of Chemical Technology, Mumbai, India.

She has over a decade of experience leading, designing, and developing test automation solutions for various platforms across healthcare, banking, and manufacturing industries using Agile and Waterfall methodologies. She is experienced in building reusable and extendable automation frameworks for web applications, REST, SOAP, and microservices. She is a strong follower of the shift-left testing approach, a certified AWS Cloud Practitioner, and a Machine Learning Specialist. She is passionate about utilizing AI-related technologies in software testing and the healthcare industry.

SRAVANTHY MYNENI received the bachelor's degree in computer science, in 2013, and the Master of Science degree in information technology and management from Illinois Institute of Technology, Chicago, IL, USA, in 2017. She is currently an Engineer focused on data engineering and analysis. She has more than eight years of experience in designing, building, and deploying data-centric solutions using Agile and Waterfall methodologies. She is enthusiastic about data analysis, data engineering, and AI applications to provide solutions for real-world problems.

UNNATI NETTUR is currently pursuing the bachelor's degree in computer science with Virginia Tech, Blacksburg, VA, USA. She possesses an avid curiosity about the constantly evolving field of technology and software development, with a particular interest in artificial intelligence. She is passionate about gaining experience in building innovative and creative solutions for current issues in the field of software engineering.

LIKHIT SAGAR GAJJA is currently pursuing the bachelor's degree in computer science with BML Munjal University, Haryana, India. He is evident in showing his passion for the dynamic field of technology and software development. His research interests include artificial intelligence, prompt engineering, and game-designing technologies, highlighting his dedication to obtaining hands-on experience and developing innovative solutions for real-time issues in software engineering.

DAVE BURKE received the bachelor's degree in economics from Providence College and the master's degree in management of information technology from the University of Virginia. He brings over 24 years of leadership and full life cycle software development experience to Coveros Inc. His experience includes work across various markets including telecommunications, media, logistics, transportation, and financial services. Before Coveros Inc., he was the COO with Command Information/Digital Focus, an industry leader in agile software development, and held leadership roles at Verizon Communications. In addition, he served seven years as an Officer in the U.S. Army.

TOM STIEHM received the B.S. degree in computer science from George Mason University. He is a 20-year veteran of the information technology industry. He has spent the past ten years managing, designing, and implementing software products and applications using agile software development methods. Before Coveros Inc., he held a variety of CTO and architect positions at software development companies. He is a member of the Northern Virginia BEA Users Group and the Northern Virginia Java Users Group.

JEFFERY PAYNE received the B.S. degree in computer science from the Allegheny College and the M.S. degree in computer science from the College of William and Mary. Since 2008, he has been leading Coveros Inc. Under his guidance, the company has become a recognized market leader in secure agile software development. Before Coveros Inc., he was the Co-Founder, the CEO, and the Chairperson of the Board of Cigital, where he led the startup and growth of the company for 16 years. He is a popular keynote and featured speaker at technology conferences and has testified before Congress on technology issues, such as intellectual property rights and cyber-terrorism. He is a former ACM National Lecturer and the Co-Founder of the Northern Virginia Chapter of the IEEE Computer Society.

• • •